

```
In [71]: import warnings
warnings.filterwarnings("ignore")
```

Time Series Analysis

Time series: is data that is collected in a specific order over time. The key difference from regular data is that the order matters.

Examples:

- Your weight measured every morning
- The number of customers visiting a store each day
- Stock price at the end of each day
- Number of steps you walk each hour

Why time series is important?

You can spot patterns.

Example: you might notice that a store always gets more customers on weekends, Once you see these patterns, you can make predictions "next Saturday will probably be busy too."

basic components to look out for:

- Trend — Is it generally going up or down? (Like sales increasing each month)
- Seasonality — Does it repeat regularly? (Like more ice cream sales during festive season)
- Noise — Random ups and downs with no clear pattern

```
In [72]: import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
```

`statsmodels` is a library in Python, which is use for statistical modeling, including time series analysis.

```
In [73]: df = pd.read_csv('/content/drive/MyDrive/Datasets/AirPassengers.csv')
```

```
In [74]: df.head()
```

```
Out[74]:
```

	Month	#Passengers
0	1949-01	112
1	1949-02	118
2	1949-03	132
3	1949-04	129
4	1949-05	121

```
In [75]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144 entries, 0 to 143
Data columns (total 2 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Month           144 non-null   object
1   #Passengers     144 non-null   int64
dtypes: int64(1), object(1)
memory usage: 2.4+ KB
```

Convert Month to datetime and set as index

`DateTime` is a data type used to store both date and time together—day, month, year, hour, minute, and second. It represents a specific moment in time.

In Python (and programming in general), `datetime` is a specialized data type that captures an exact point in time by combining date and time information into a single value.

Why is `datetime` important in time series?

Plots make sense:

- If you keep dates as text, Python won't know it's sequential.
- You can't create proper time-based plots.

Resampling and rolling calculations:

- Example: "Show average passengers per year instead of per month"
- Requires `datetime` to group by time intervals.

Time-based indexing:

- You can easily select data

Forecasting depends on order:

- Time series models need real sequential dates, not strings, to predict future points.

```
In [76]: df['Month'] = pd.to_datetime(df['Month'])
df.set_index('Month', inplace=True)
```

```
In [77]: df.head()
```

```
Out[77]:
```

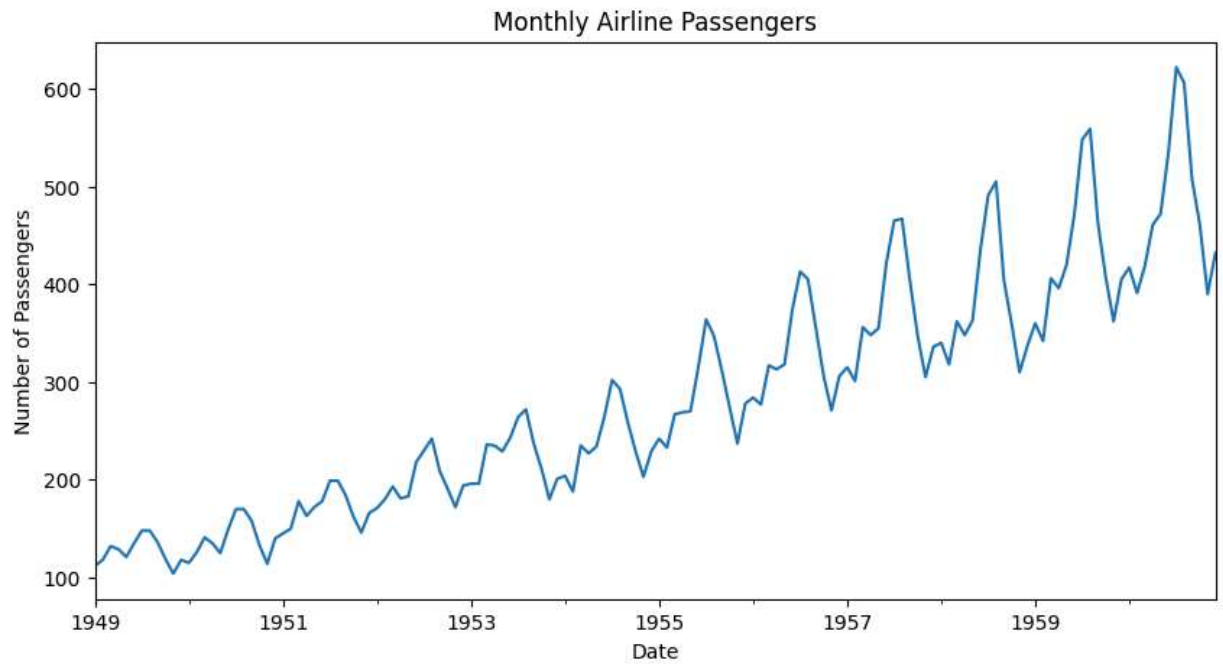
	#Passengers
Month	
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

```
In [78]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Data columns (total 1 columns):
#   Column          Non-Null Count  Dtype
---  -
0   #Passengers      144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB
```

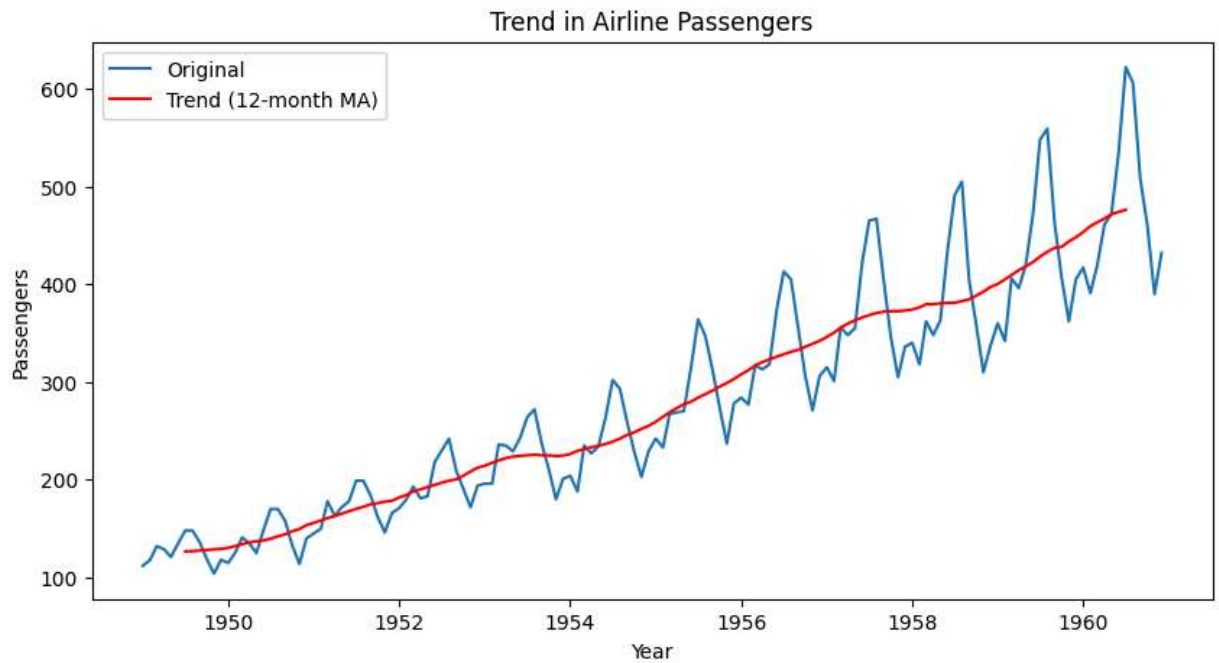
Plot the series to show trend and seasonality

```
In [79]: df['#Passengers'].plot(title="Monthly Airline Passengers", figsize=(10,5))  
plt.xlabel("Date")  
plt.ylabel("Number of Passengers")  
plt.show()
```



```
In [80]: df['Trend'] = df['#Passengers'].rolling(window=12, center=True).mean()

plt.figure(figsize=(10,5))
plt.plot(df['#Passengers'], label='Original')
plt.plot(df['Trend'], label='Trend (12-month MA)', color='red')
plt.title("Trend in Airline Passengers")
plt.xlabel("Year")
plt.ylabel("Passengers")
plt.legend()
plt.show()
```



```
In [81]: decomposition = seasonal_decompose(df['#Passengers'], model='multiplicative', per

trend = decomposition.trend
seasonal = decomposition.seasonal
residual = decomposition.resid

plt.figure(figsize=(10,8))

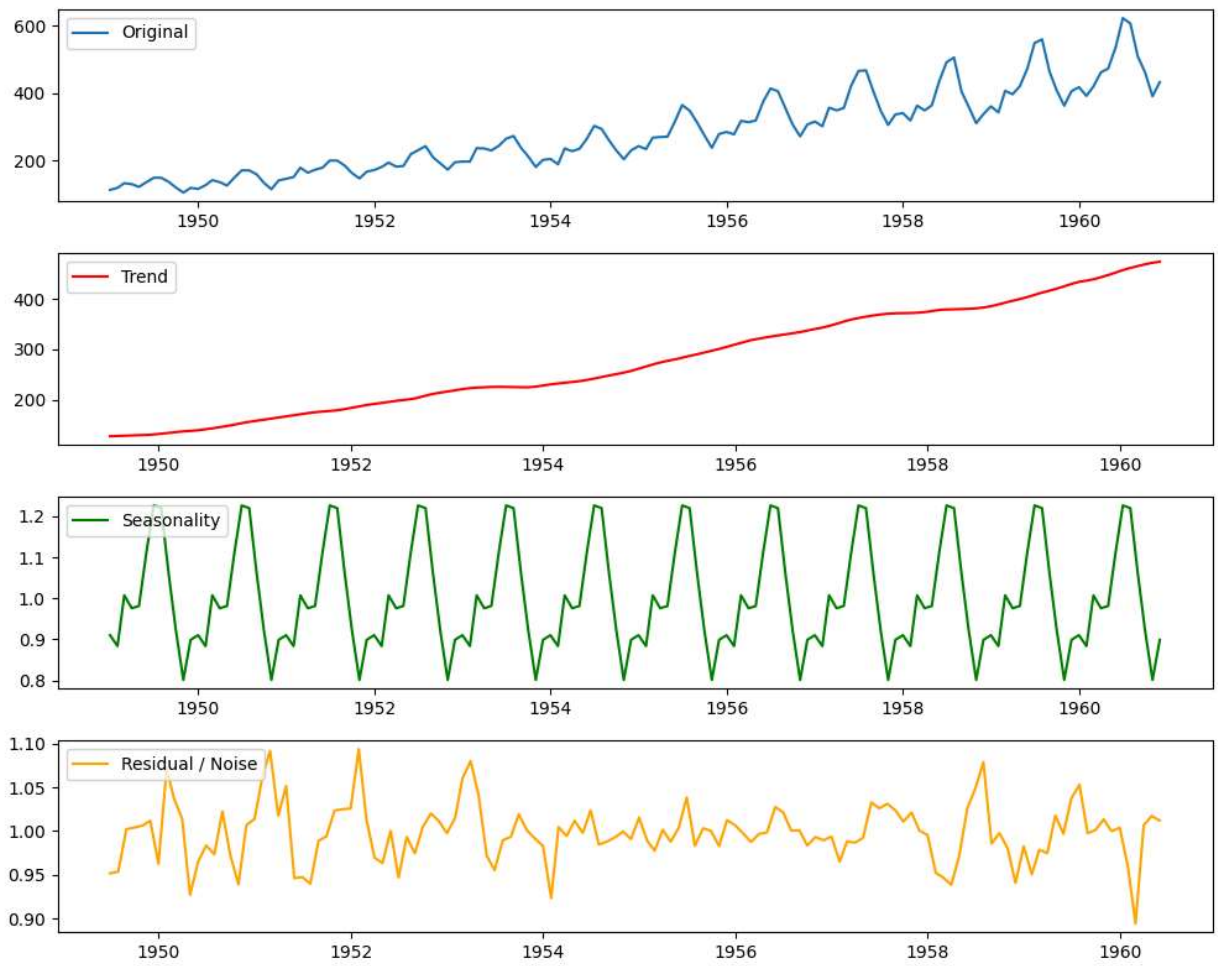
plt.subplot(4,1,1)
plt.plot(df['#Passengers'], label='Original')
plt.legend(loc='upper left')

plt.subplot(4,1,2)
plt.plot(trend, label='Trend', color='red')
plt.legend(loc='upper left')

plt.subplot(4,1,3)
plt.plot(seasonal, label='Seasonality', color='green')
plt.legend(loc='upper left')

plt.subplot(4,1,4)
plt.plot(residual, label='Residual / Noise', color='orange')
plt.legend(loc='upper left')

plt.tight_layout()
plt.show()
```



##Stationary and Non-Stationary Time Series

Stationary Time Series A stationary time series is one where the data stays relatively stable over time. The pattern doesn't change.

Characteristics:

- No strong upward or downward trend
- No seasonality (repeating patterns)

Non-Stationary Time Series A non-stationary time series is one where the data changes over time. The pattern is not stable.

Characteristics:

- Has a clear trend (going up or down)
- Has seasonality (repeating patterns)

Why Does This is important

Most forecasting methods (like ARIMA) require stationary data to work properly. If your data is non-stationary, you need to transform it first, remove the trend or seasonality to make it stationary before using those methods.

##Tests for Stationarity

Augmented Dickey-Fuller (ADF) Test This is the most common test.**How it works:**

- It checks if the time series has a unit root (a mathematical way of saying "non-stationary")
- It gives you a p-value (a number between 0 and 1)

interpretation:

- p-value < 0.05 → Data is stationary (reject the trend)
- p-value ≥ 0.05 → Data is non-stationary (has a trend)

```
In [82]: from statsmodels.tsa.stattools import adfuller

result = adfuller(df['#Passengers'])

print('ADF Statistic:', result[0])
print('p-value:', result[1])
if result[1] < 0.05:
    print("The series is likely stationary")
else:
    print("The series is likely non-stationary")
```

```
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
The series is likely non-stationary
```

##How to Make Data Stationary

If the time series is non-stationary, We transform the data so its behavior becomes stable over time.

common methods

- Differencing

Instead of looking at the actual value, we look at how much it changes from one time step to the next.

- Detrending

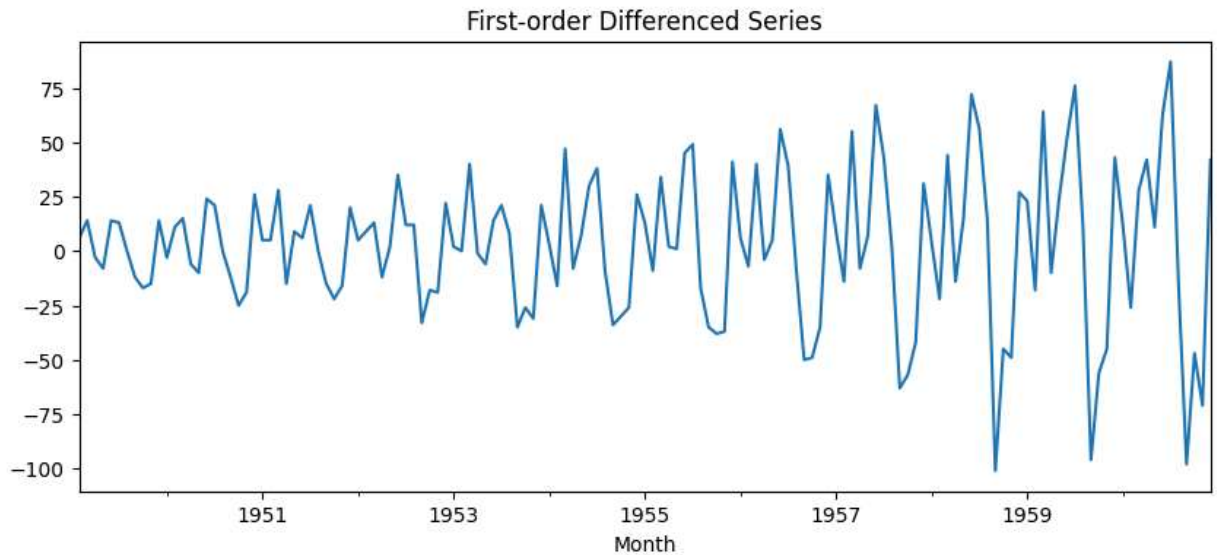
Remove the trend line mathematically: dividing the observed variable by trend

```
df['Detrended_Ratio'] = df['#Passengers'] / df['Trend']
```

```
In [83]: df['Diff_1'] = df['#Passengers'] - df['#Passengers'].shift(1)

#Drop missing values
df_diff = df['Diff_1'].dropna()

df_diff.plot(title="First-order Differenced Series", figsize=(10,4))
plt.show()
```



```
In [84]: result_diff = adfuller(df_diff)

print('ADF Statistic:', result_diff[0])
print('p-value:', result_diff[1])
if result_diff[1] < 0.05:
    print("The series is likely stationary")
else:
    print("The series is likely non-stationary")
```

ADF Statistic: -2.8292668241700047

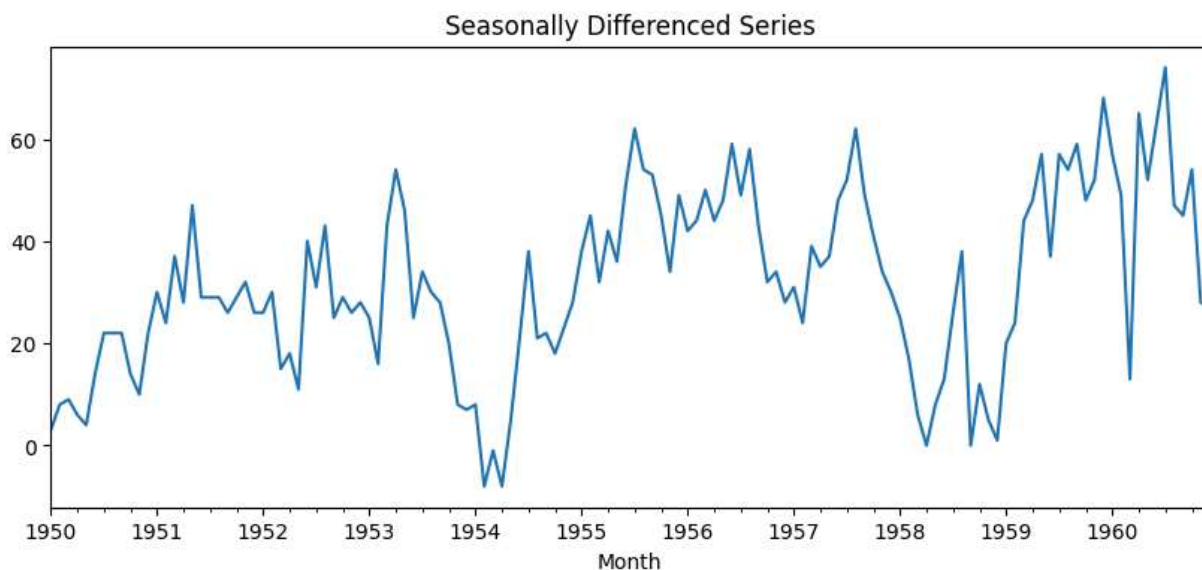
p-value: 0.05421329028382478

The series is likely non-stationary

Introduce seasonal differencing

```
In [85]: df['Seasonal_Diff'] = df['#Passengers'] - df['#Passengers'].shift(12)
df_seasonal = df['Seasonal_Diff'].dropna()

df_seasonal.plot(title="Seasonally Differenced Series", figsize=(10,4))
plt.show()
```



```
In [86]: result = adfuller(df_seasonal)

print('ADF Statistic:', result[0])
print('p-value:', result[1])
if result[1] < 0.05:
    print("The series is likely stationary")
else:
    print("The series is likely non-stationary")
```

```
ADF Statistic: -3.383020726492481
p-value: 0.011551493085514952
The series is likely stationary
```

#What Is ARIMA?

Now that our data is stationary, we can use a model called ARIMA.

ARIMA: is a time-series forecasting method that learns from the past to predict the future.

The name ARIMA tells us exactly how it works.

AR — AutoRegressive

Auto = self

Regressive = using past values

- AR means the model uses previous values of the same data to make predictions.

I — Integrated

We modified the data before modeling it.

Specifically:

- We removed trend
- We removed seasonality
- We made the data stationary

Example: Instead of modeling raw passenger counts, we model changes in passenger counts.

MA — Moving Average

This does NOT mean the average of values.

- MA means the model looks at past prediction errors.

“If the model under-predicted last time, it adjusts the next prediction.”

So:

MA learns from past mistakes.

Put Together: ARIMA

When we combine everything:

- AR → learns from past values
- I → uses stationary (differenced) data
- MA → corrects past errors

#ACF & PACF: Understanding Patterns in Time Series

Before fitting ARIMA, we need to decide how many past values and past errors to include.

- ACF and PACF are the tools that help us do this.

Autocorrelation (ACF)

It shows how today's value is related to past values. If past values are similar, they can help predict today.

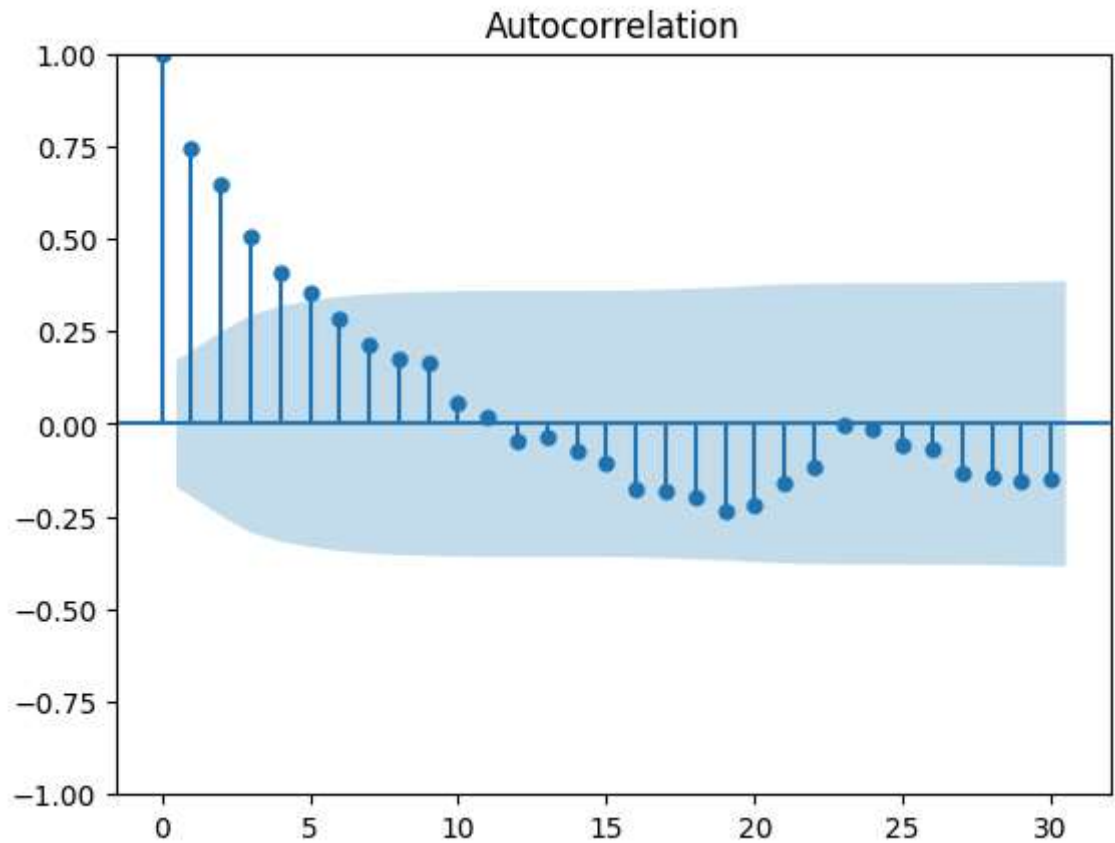
Partial Autocorrelation (PACF)

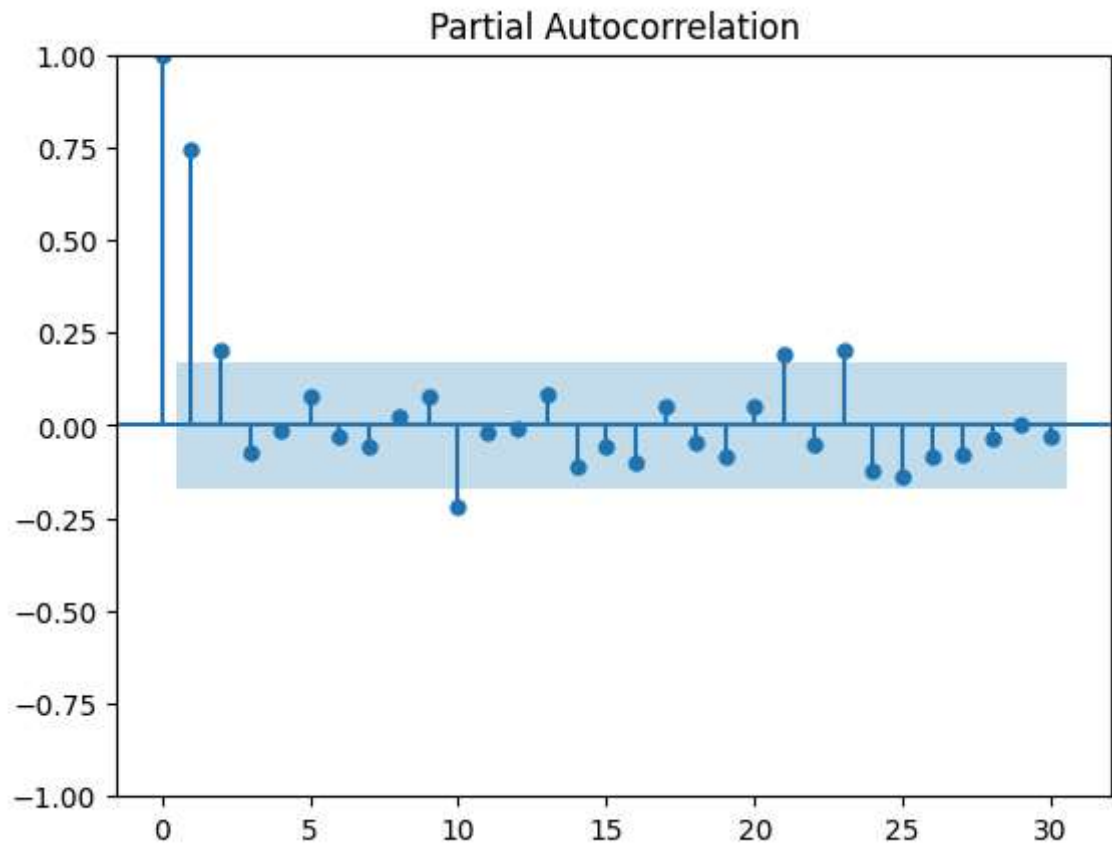
It shows how today's value is related to a specific past value, ignoring the influence of values in between.

```
In [87]: from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

plot_acf(df_seasonal, lags=30)
plt.show()

plot_pacf(df_seasonal, lags=30)
plt.show()
```





##How splitting is different from regular ML

In regular ML:

- We often shuffle the data and split randomly
- This is fine when the order doesn't matter

In time series:

- Order matters
- We must always split by time, not randomly

Otherwise, the model could see a future month while trying to predict an earlier month, giving unrealistically good results.

```
In [88]: train_size = int(len(df) * 0.8)

train = df['#Passengers'][:train_size]
test = df['#Passengers'][train_size:]
```

```
In [89]: train.tail()
```

```
Out[89]:
```

	#Passengers
Month	
1958-03-01	362
1958-04-01	348
1958-05-01	363
1958-06-01	435
1958-07-01	491

dtype: int64

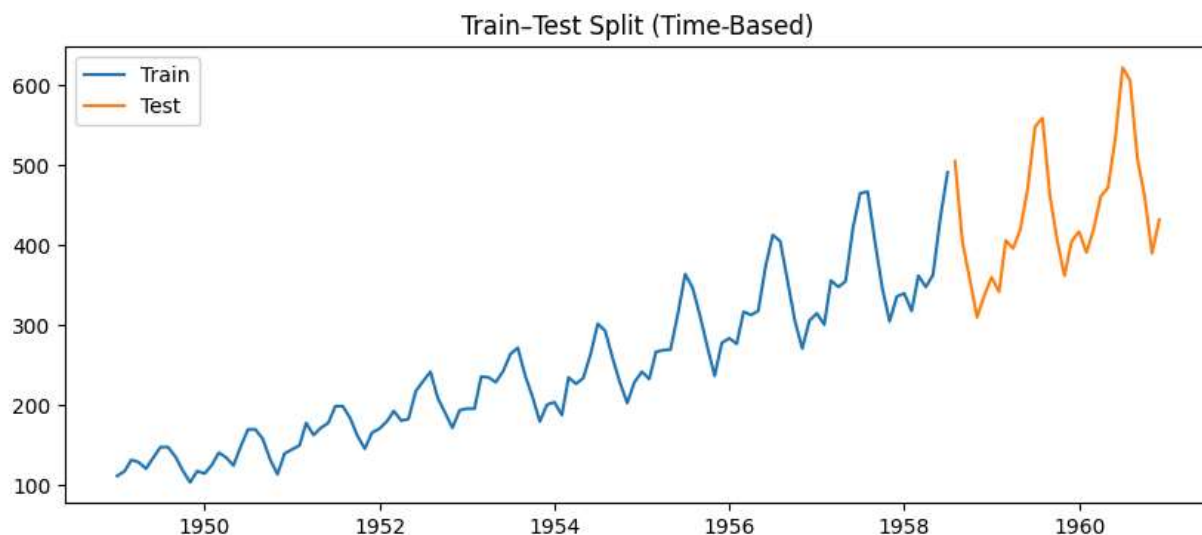
```
In [90]: test.head()
```

```
Out[90]:
```

	#Passengers
Month	
1958-08-01	505
1958-09-01	404
1958-10-01	359
1958-11-01	310
1958-12-01	337

dtype: int64

```
In [91]: plt.figure(figsize=(10,4))
plt.plot(train, label='Train')
plt.plot(test, label='Test')
plt.legend()
plt.title("Train-Test Split (Time-Based)")
plt.show()
```



```
In [92]: from statsmodels.tsa.arima.model import ARIMA

model = ARIMA(train, order=(1, 1, 3))
fitted_model = model.fit()
```

What does order = (p, d, q) mean in ARIMA?

this are ARIMA control knobs, Each one answers a different question about the data.

p — AutoRegressive (AR) part

“How many past values do we look at?”

- $p = 1 \rightarrow$ today depends on yesterday
- $p = 2 \rightarrow$ today depends on last two days

This is about memory in the series

Decided mainly using PACF

d — Differencing

“How many times do we difference the data to remove trend?”

- $d = 1 \rightarrow$ subtract previous value once
- $d = 0 \rightarrow$ data is already stationary
- $d = 2 \rightarrow$ difference twice

q — Moving Average (MA) part

“How many past errors do we use?”

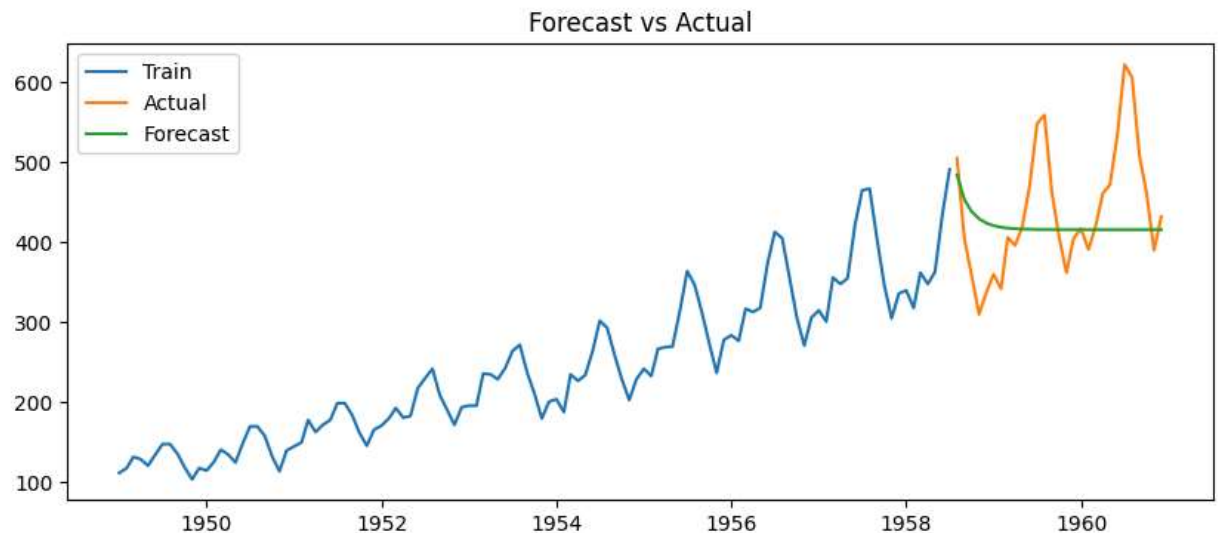
- $q = 1 \rightarrow$ model uses yesterday's error

This helps correct past mistakes

Decided mainly using ACF

```
In [93]: forecast_1 = fitted_model.forecast(steps=len(test))

plt.figure(figsize=(10,4))
plt.plot(train, label='Train')
plt.plot(test, label='Actual')
plt.plot(forecast_1, label='Forecast')
plt.legend()
plt.title("Forecast vs Actual")
plt.show()
```



Why ARIMA Failed

ARIMA failed not because it is wrong, but because it does not understand seasons patterns.

- ARIMA has short-term memory.
- It remembers yesterday, but it forgets last year

SARIMA

SARIMA adds seasonality to ARIMA.

Seasonal patterns repeat over a fixed period (e.g., monthly, quarterly). SARIMA adds four parameters:

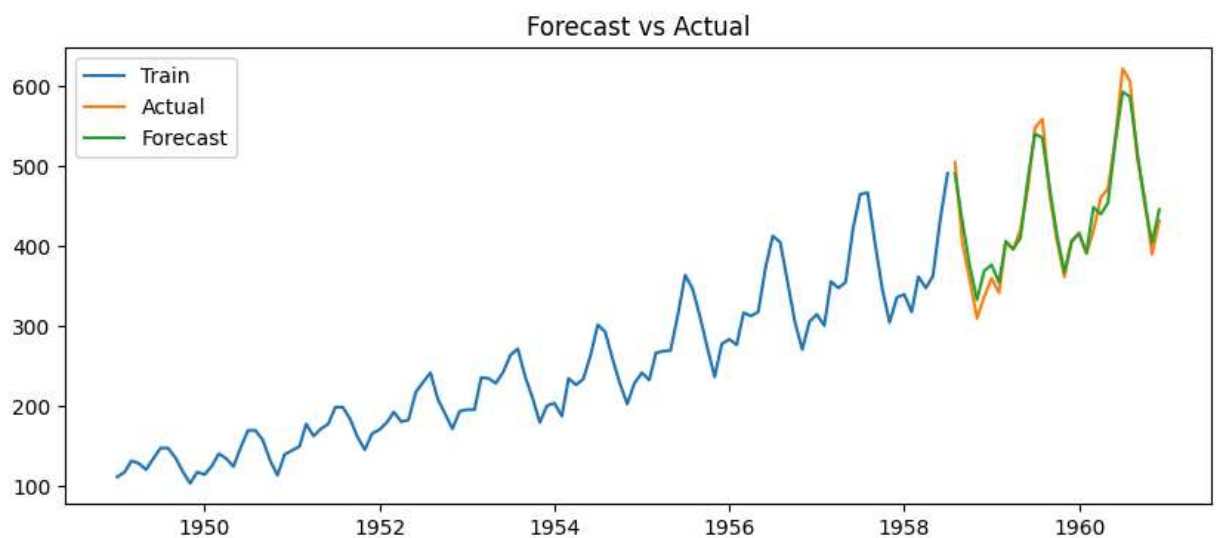
- P, D, Q – the seasonal counterparts of p, d, q
- s – the length of the seasonal cycle (e.g., 12 for monthly data with yearly seasonality)

```
In [94]: from statsmodels.tsa.statespace.sarimax import SARIMAX
```

```
model = SARIMAX(  
    train,  
    order=(1,1,3),  
    seasonal_order=(1,1,3,12)  
)  
  
results = model.fit()
```

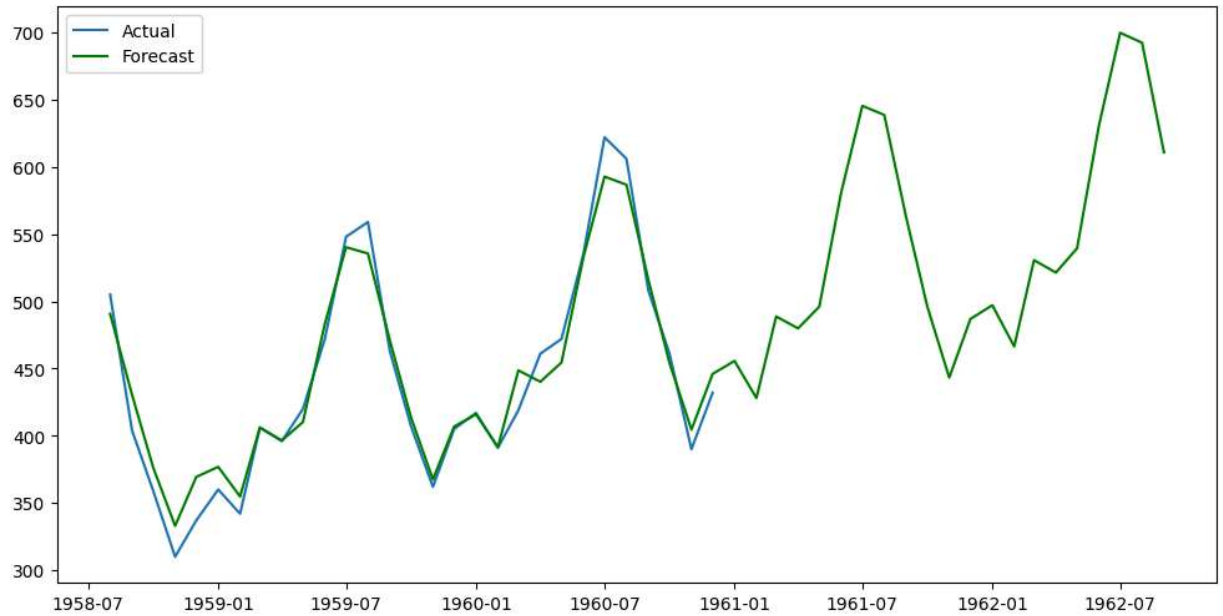
```
In [96]: forecast_2 = results.forecast(steps=len(test))
```

```
plt.figure(figsize=(10,4))  
plt.plot(train, label='Train')  
plt.plot(test, label='Actual')  
plt.plot(forecast_2, label='Forecast')  
plt.legend()  
plt.title("Forecast vs Actual")  
plt.show()
```



```
In [98]: forecast = results.get_forecast(steps=36)

plt.figure(figsize=(12, 6))
plt.plot(test, label='Actual')
plt.plot(forecast_values, label='Forecast', color='green')
plt.legend()
plt.show()
```



In [97]: